ПМ3 Разработка модулей ПО.

РО 3.1 Понимать и применять принципы объектно-ориентированного и асинхронного программирования.

Тема 1. Введение в ООП.

Лекция 5. Классы в JavaScript: основы.

Цель занятия:

Познакомиться с современным синтаксисом классов, понять их связь с прототипами и преимущества использования.

Учебные вопросы:

1. Что такое класс. Синтаксис объявления.
2. Конструктор и ключевое слово this.
3. Поля класса — внутри и вне конструктора.
4. Методы класса и прототип.
5. Создание экземпляров через new.
6. Отличие классов от функций-конструкторов.

1. Понятие класса.

Класс в JavaScript — это шаблон для создания объектов. Он позволяет организовать код более структурировано и удобно, особенно при работе с объектно-ориентированным программированием.

Классы в JavaScript можно рассматривать как «синтаксический сахар» над функциями-конструкторами и прототипами.

Это означает, что они предоставляют более удобный и понятный способ создания объектов и работы с их свойствами и методами.

Преимущества:

- Более читаемый и понятный синтаксис.
- Автоматическое создание прототипов для методов

До появления классов объекты в JavaScript создавались через функции-конструкторы и прототипы:

```
function Company(name, website) {
    this.name = name;
    this.website = website;
}

Company.prototype.getName = function() {
    return this.name;
}

const c = new Company("Google", "google.com");
```

Это работает, но код выглядит громоздко, особенно когда методов много. Приходится отдельно писать функцию, отдельно добавлять методы в prototype.

Синтаксис классов.

В ES6 (2015) появился новый синтаксис — классы. Класс объявляется с помощью ключевого слова **class**, за которым следует его имя по соглашению, с заглавной буквы (PascalCase).

```
class Company {
    // Тело класса
}
```

Два способа объявить класс:

1. Через объявление (Class Declaration) — самый распространённый способ:

```
class Company {
    // тело класса
}
```

2. Через выражение (Class Expression) — класс присваивается переменной. Используется реже, например когда класс нужно передать как аргумент или вернуть из функции:

```
const Company = class {
    // тело класса
};
```

Ключевое отличие от функций: нет **hoisting**

Функции в JavaScript «поднимаются» — их можно вызвать до объявления в коде. С классами так не работает:

```
const c = new Company(); // ❌ ReferenceError
class Company {}
```

Класс нужно объявить до его использования.

Пример: класс вместо функции-конструктора

```
class Company {
    constructor(name, website) {
        this.name = name;
        this.website = website;
    }

    getName() {
        return this.name;
    }
}

const c = new Company("Google", "google.com");
console.log(c.getName()); // "Google"
```

Результат тот же, но код читается как единый блок: сразу видно, что у объекта есть поля name и website, и метод getName. Всё в одном месте.

Почему классы лучше для больших проектов?

Когда объектов много и у каждого десятки методов — разбросанные prototype-записи превращаются в хаос.

Класс держит всё вместе: свойства, методы, логику инициализации. Это удобнее читать, поддерживать и передавать другому разработчику.

Плюс синтаксис классов знаком тем, кто приходит из Java, Python, C# — не нужно заново выстраивать модель мышления.

Итог:

- Класс — шаблон для создания объектов
- Это синтаксический сахар над прототипами, не новый механизм
- Объявляется через class ИмяКласса {}
- Имя — с заглавной буквы
- Не поднимается (no hoisting) — объявляй до использования
- Код становится структурированным и читаемым

2. Конструктор и ключевое слово this.

Что такое конструктор?

Когда мы создаём объект через **new**, JavaScript должен знать — какие значения записать в свойства нового объекта. Именно для этого существует специальный метод внутри класса — **constructor**.

constructor вызывается автоматически в момент создания экземпляра. Писать его вызов вручную не нужно и нельзя.

Пример:

```
class Company {
    constructor(name, website) {
        this.name = name;
        this.website = website;
    }
}

const c = new Company("Google", "google.com");
// constructor вызвался сам, name и website записались в объект
```

Что такое this?

Внутри конструктора (и любого метода класса) ключевое слово **this** ссылается на тот конкретный объект, который сейчас создаётся или вызывает метод.

Проще говоря — **this** это «я сам». Когда мы пишем this.name = name, мы говорим: «запиши значение name как свойство этого объекта».

Пример:

```
class Company {
    constructor(name) {
        this.name = name; // this = создаваемый объект
    }
}

const c1 = new Company("Google");
const c2 = new Company("Apple");

console.log(c1.name); // "Google"
console.log(c2.name); // "Apple"
//Каждый раз при вызове new создаётся новый объект,
// и this указывает именно на него.
// Поэтому c1 и c2 независимы друг от друга
```

Параметры конструктора

Конструктор — обычный метод, он принимает аргументы так же, как любая функция. Аргументы передаются при вызове **new**:

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
}

const p = new Person("Иван", 22);
console.log(p.name); // "Иван"
console.log(p.age);  // 22
```

Конструктор необязателен

Если конструктор не написан явно — подставит пустой автоматически. Объект создастся без начальных свойств:

```
class Company {
    // конструктор не написан — подставится пустой
}

const c = new Company();
console.log(c); // Company {}
```

Это нормально, если свойства планируется задавать позже или через поля класса.

В классе может быть только один конструктор. Попытка написать два **constructor** — это ошибка.

Если нужны разные сценарии создания объекта — используют значения по умолчанию:

```
class Company {
    constructor(name, website = "не указан") {
        this.name = name;
        this.website = website;
    }
}

const c1 = new Company("Google", "google.com");
const c2 = new Company("Стартап"); // website = "не указан"

console.log(c1.website); // "google.com"
console.log(c2.website); // "не указан"
```

Что происходит при вызове **new** — шаг за шагом

Когда пишем new Company("Google"), делает следующее:

- Создаёт новый пустой объект {}
- Устанавливает ему прототип — Company.prototype
- Вызывает constructor, передав этот объект как this
- Выполняет тело конструктора — записывает свойства
- Возвращает готовый объект

Всё это происходит автоматически. Нам не нужно ничего возвращать из конструктора — return там не пишут.

this вне класса ведёт себя иначе

Важно понимать: **this** в зависит от контекста вызова. Внутри класса всё предсказуемо. Но если метод класса передать как колбэк — **this** может потеряться:

```
class Timer {
    constructor() {
        this.seconds = 0;
    }

    start() {
        // ❌ this здесь будет undefined (или window в браузере)
        setInterval(function() {
            this.seconds++;
        }, 1000);

        // ✅ стрелочная функция сохраняет this из внешнего контекста
        setInterval(() => {
            this.seconds++;
        }, 1000);
    }
}
```

Это частая ошибка начинающих.

Обычная функция (function() {}) сама определяет свой this в момент вызова — в зависимости от того, как она вызвана, а не где написана.

Стрелочная функция не имеет своего **this** — она берёт его из того места, где была создана.

Поэтому внутри классов в колбэках всегда используют стрелочные функции.

Итог:

- **constructor** — специальный метод, вызывается автоматически при new
- **this** внутри класса — ссылка на создаваемый или вызывающий объект
- Конструктор может принимать параметры, поддерживает значения по умолчанию
- В классе допускается только один constructor
- В колбэках внутри методов **this** может теряться — используй стрелочные функции

3. Поля класса — внутри и вне конструктора.

Два способа задать свойства объекта

В современном JavaScript свойства класса (поля) можно объявить двумя способами:

- внутри конструктора через this
- вне конструктора — прямо в теле класса

Оба способа рабочие. Разница — в удобстве и назначении.

Способ 1 — через конструктор (классический)

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
}
```

const p = new Person("Иван", 22);

console.log(p.name); // "Иван"

Свойства создаются в момент вызова new и получают значения из аргументов. Если аргумент не передан — свойство будет undefined.

Способ 2 — поля класса (современный синтаксис)

можно объявлять поля прямо в теле класса, до конструктора:

```
class Person {
    name;         // поле без значения — будет undefined
    role = "Guest";   // поле со значением по умолчанию
    salary = 100000;  // поле со значением по умолчанию

    constructor(name) {
        this.name = name; // переопределяем значение
    }
}

const p = new Person("Мария");
console.log(p.name);   // "Мария"
console.log(p.role);   // "Guest"
console.log(p.salary); // 100000
```

Поля объявляются до конструктора, сразу видно — какие данные хранит объект. Это улучшает читаемость, особенно в больших классах.

Как они работают вместе

Поля класса инициализируются до выполнения тела конструктора. То есть порядок такой:

- Создаётся объект
- Полям присваиваются значения по умолчанию
- Выполняется конструктор — может переопределить значения

Пример:

```
class Person {
    role = "Guest";

    constructor(role) {
        // если role передан — переопределяем, иначе оставляем "Guest"
        this.role = role ?? this.role;
    }
}

const p1 = new Person("Admin");
const p2 = new Person();

console.log(p1.role); // "Admin"
console.log(p2.role); // "Guest"
```

Оператор **??** — это «нулевое слияние». Он говорит: если левая часть null или undefined — возьми правую. Иначе оставь левую.

Когда использовать поля вне конструктора

Поля вне конструктора удобны в трёх случаях:

1. Значение по умолчанию, которое не зависит от аргументов:

```
class Car {
    speed = 0;        // машина всегда начинает с нуля
    isRunning = false;

    constructor(brand) {
        this.brand = brand;
    }
}
```

2. Когда хочется сразу показать структуру объекта:

```
class User {
    name;
    email;
    isActive = true;
    role = "user";

    constructor(name, email) {
        this.name = name;
        this.email = email;
    }
}
```

Читая класс сверху вниз, сразу понятно — у User есть 4 поля. Не нужно искать их внутри конструктора.

3. Поля с фиксированным значением, не зависящим от экземпляра — но для этого лучше использовать **static** (разберём в сл. лекции).

Итог:

- Свойства объекта можно задавать внутри constructor через **this** или прямо в теле класса
- Поля вне конструктора инициализируются до его выполнения
- Поля вне конструктора улучшают читаемость — структура объекта видна сразу
- Значения по умолчанию удобно задавать через поля класса, а переопределять в конструкторе
- Каждый экземпляр получает свою независимую копию полей

4. Методы класса.

Метод — это функция, которая принадлежит объекту и описывает его поведение.

В классе методы объявляются прямо в теле, без ключевых слов **function** или **const**.

Пример:

```
class Company {
    constructor(name, website) {
        this.name = name;
        this.website = website;
    }
    // методы класса
    getName() {
        return this.name;
    }

    getWebsite() {
        return this.website;
    }
}

const c = new Company("Google", "google.com");
console.log(c.getName());    // "Google"
console.log(c.getWebsite()); // "google.com"
```

Внутри метода **this** работает так же, как в конструкторе — ссылается на объект, который вызвал метод.

Методы не видны при выводе объекта

Это часто удивляет новичков. Если вывести объект в консоль — методов там не будет:

```
class Company {
    constructor(name) {
        this.name = name;
    }
    getName() { return this.name; }
}

const c = new Company("Google");
console.log(c); // Company { name: "Google" }
```

Метод getName не показан — потому что он находится не в объекте, а в прототипе.

Методы одного класса могут обращаться друг к другу через **this**:

```
class Person {
    constructor(firstName, lastName, age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
    }

    getFullName() {
        return `${this.firstName} ${this.lastName}`;
    }

    getInfo() {
        // вызываем другой метод класса через this
        return `${this.getFullName()}, ${this.age} лет`;
    }
}

const p = new Person("Иван", "Иванов", 22);
console.log(p.getInfo()); // "Иван Иванов, 22 лет"
```

Методы vs поля — где что хранится

```
class Student {
    score = 0;          // поле — хранится в объекте

    constructor(name) {
        this.name = name; // поле — хранится в объекте
    }

    getInfo() { return `${this.name}: ${this.score}`; }  // метод — в прототипе
    addScore(n) { this.score += n; }                      // метод — в прототипе
}

const s = new Student("Иван");
s.addScore(90);
console.log(s.getInfo()); // "Иван: 90"
// что хранится в самом объекте:
console.log(s); // Student { score: 90, name: "Иван" }
// что хранится в прототипе:
console.log(Object.getOwnPropertyNames(Student.prototype));
// [ 'constructor', 'getInfo', 'addScore' ]
```

Итог:

- Методы объявляются в теле класса без function и const
- Внутри метода **this** — объект, который вызвал метод
- Методы автоматически помещаются в прототип класса — один раз для всех экземпляров
- Это экономит память: не копия метода в каждом объекте, а одна функция в прототипе
- При вызове объект.метод() ищет метод сначала в объекте, затем в прототипе
- Методы могут вызывать друг друга через this

5. Создание экземпляров через new.

Что такое экземпляр?

Класс — это шаблон. Сам по себе он ничего не делает и никаких данных не хранит. Чтобы получить реальный объект — нужно создать экземпляр класса.

Экземпляр — это конкретный объект, созданный по шаблону класса. Можно создать сколько угодно экземпляров одного класса — каждый будет независимым объектом со своими данными.

Оператор new

Экземпляр создаётся с помощью оператора **new**:

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    getInfo() {
        return `${this.name}, ${this.age} лет`;
    }
}

const p1 = new Person("Иван", 22);
const p2 = new Person("Мария", 25);

console.log(p1.getInfo()); // "Иван, 22 лет"
console.log(p2.getInfo()); // "Мария, 25 лет"
//p1 и p2 — два независимых объекта. Изменение одного не затрагивает другой.
```

Что делает **new** шаг за шагом

```
const p = new Person("Иван", 22);
```

- Создаётся новый пустой объект {}
- Ему устанавливается прототип — Person.prototype
- Конструктор вызывается с этим объектом как this
- Выполняется тело конструктора — записываются свойства
- Готовый объект возвращается и присваивается переменной p

Всё это происходит автоматически при каждом вызове new.

Без **new** — ошибка

Класс нельзя вызвать как обычную функцию:

```
const p = Person("Иван", 22);
// ❌ TypeError: Class constructor Person cannot be invoked without 'new'
```

Это одно из ключевых отличий класса от функции-конструктора — защита от случайного вызова без new.

Проверка принадлежности — **instanceof**

Оператор instanceof позволяет проверить, является ли объект экземпляром определённого класса:

```
const p = new Person("Иван", 22);

console.log(p instanceof Person); // true
console.log(p instanceof Object); // true — все объекты наследуют от Object
```

Это полезно когда нужно убедиться, что функция получила объект нужного типа:

```
function printInfo(obj) {
    if (!(obj instanceof Person)) {
        console.error("Ожидается объект Person");
        return;
    }
    console.log(obj.getInfo());
}

printInfo(p);                // "Иван, 22 лет"
printInfo({ name: "X"}); // Ожидается объект Person
```

Экземпляр можно создать без аргументов

Если конструктор не требует аргументов или все они имеют значения по умолчанию:

```
class Counter {
    count = 0;
    increment() { this.count++; }
    reset() { this.count = 0; }
    getValue() { return this.count; }
}

const counter = new Counter(); // без аргументов
counter.increment();
counter.increment();
counter.increment();
console.log(counter.getValue()); // 3
counter.reset();
console.log(counter.getValue()); // 0
```

Экземпляры можно хранить в массиве

На практике часто создают несколько объектов одного класса и работают с ними как с коллекцией:

```
class Student {
    constructor(name, score) {
        this.name = name;
        this.score = score;
    }
}

const students = [
    new Student("Иван", 90),
    new Student("Мария", 85),
    new Student("Анна", 78),
];
```

```
// найти лучшего студента
const best = students.reduce((top, s) => s.score > top.score ? s : top);
console.log(best.name); // "Иван"

// вывести всех
students.forEach(s => console.log(`${s.name}: ${s.score}`));
// Иван: 90
// Мария: 85
// Анна: 78
```

Это типичный паттерн — класс описывает одну сущность, массив хранит коллекцию таких сущностей.

Итог:

- Экземпляр — конкретный объект, созданный по шаблону класса
- Создаётся через new ИмяКласса(аргументы)
- new создаёт объект, устанавливает прототип, вызывает конструктор
- Без new класс вызвать нельзя — будет ошибка
- instanceof проверяет принадлежность объекта классу
- Данные у каждого экземпляра свои, методы — общие из прототипа
- Экземпляры удобно хранить в массивах и работать с ними как с коллекцией

6. Отличие классов от функций-конструкторов.

В реальных проектах и на собеседованиях часто встречается старый код — написанный через функции-конструкторы и прототипы.

Понимание разницы помогает читать такой код, не теряться и понимать, что классы — это не магия, а просто удобная обёртка над тем же механизмом.

Напишем одно и то же двумя способами:

Через функцию-конструктор (старый способ):

```
function Person(name, age) {
    this.name = name;
    this.age = age;
}

Person.prototype.getInfo = function() {
    return `${this.name}, ${this.age} лет`;
};

const p = new Person("Иван", 22);
console.log(p.getInfo()); // "Иван, 22 лет"
```

Через класс (современный способ)

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    getInfo() {
        return `${this.name}, ${this.age} лет`;
    }
}

const p = new Person("Иван", 22);
console.log(p.getInfo()); // "Иван, 22 лет"
```

Результат одинаковый. Под капотом — тот же механизм прототипов. Но код с классом читается значительно проще.

Итоговое сравнение:

	Функция-конструктор	Класс
Синтаксис	Разбросан по файлу	Всё в одном блоке
Вызов без `new`	Тихая ошибка	TypeError сразу
Hoisting	Да	Нет
Строгий режим	Нет (если не указать)	Всегда
Приватные поля	Имитация через замыкание	Встроено (`#`)
Под капотом	Прототипы	Те же прототипы

Итоги лекции:

Класс — шаблон для создания объектов. Объявляется через class ИмяКласса {}

Класс — это синтаксический сахар над функциями-конструкторами и прототипами. Под капотом ничего не изменилось

constructor вызывается автоматически при new, инициализирует свойства объекта через this

this внутри класса — ссылка на конкретный объект, который создаётся или вызывает метод

Поля можно объявлять внутри конструктора через this и вне — прямо в теле класса

Поля вне конструктора инициализируются до его выполнения, удобны для значений по умолчанию

Каждый экземпляр получает свою независимую копию всех полей

Методы объявляются в теле класса без function. Автоматически помещаются в прототип — один раз для всех экземпляров

Экземпляр создаётся через new. Без new — TypeError

Контрольные вопросы:

- Чем класс отличается от обычной функции-конструктора?
- Что делает constructor? Обязательно ли его писать?
- Куда попадают методы, объявленные в классе?
- Можно ли вызвать класс без new?
- Как задать значение свойства по умолчанию?

Домашнее задание:

https://ru.hexlet.io/programs/js-oop

Материалы лекций:

https://github.com/ShViktor72/Education2025

Обратная связь:

colledge20education23@gmail.com